

Khant Thu Aung / 105292912

6.1P (Iteration04)

Command.cs

```
namespace SwinAdventure;
```

```
public abstract class Command : IdentifiableObject
{
    public Command(string[] ids) : base(ids) { }
    public abstract string Execute(Player p, string[] text);
}
```

LookCommand.cs

```
namespace SwinAdventure;
```

```
public class LookCommand : Command
{

    public LookCommand() : base(new string[] { "look" }) { }
    public override string Execute(Player p, string[] text)
    {
        if (text[0].ToLower() != "look")
        {
            return "Error in look input";
        }
        if (text.Length != 3 && text.Length != 5)
        {
```

```

        return "I don't know how to look like that";
    }
    if (text[1].ToLower() != "at")
    {
        return "What do you want to look at?";
    }
    if (text.Length == 5 && text[3].ToLower() != "in")
    {
        return "What do you want to look in?";
    }
    string itemId = text[2];
    IHAVEInventory container;

    if (text.Length == 3)
    {
        container = p;
    }
    else
    {
        string containerId = text[4];
        container = FetchContainer(p, containerId);
        if (container == null)
        {
            return $"I can't find the {containerId}";
        }
    }
}

```

```

        return LookAtIn(itemId, container);
    }

    private IHaveInventory FetchContainer(Player p, string containerId)
    {
        GameObject obj = p.Locate(containerId);
        return obj as IHaveInventory;
    }

    private string LookAtIn(string thingId, IHaveInventory container)
    {
        GameObject item = container.Locate(thingId);
        if (item == null)
        {
            return $"I can't find the {thingId}";
        }
        return item.FullDescription;
    }
}

```

IHaveInventory.cs

```

namespace SwinAdventure
{
    public class Inventory
    {
        private List<Item> _items;

        public Inventory()

```

```
{
    _items = new List<Item>();
}

public bool HasItem(string id)
{
    return _items.Any(_item => _item.AreYou(id));
}

public void Put(Item item)
{
    _items.Add(item);
}

public Item Take(string id)
{
    for (int i = 0; i < _items.Count; i++)
    {
        if (_items[i].AreYou(id))
        {
            Item takenItem = _items[i];
            _items.RemoveAt(i);
            return takenItem;
        }
    }

    return null;
}

public Item Fetch(string id)
{

```

```

        foreach (Item item in _items)
        {
            if (item.AreYou(id)) return item;
        }

        return null;
    }

    public string ItemList
    {
        get
        {
            string list = "";

            foreach (Item item in _items)
            {
                list += item.ShortDescription + "\n";
            }

            return list.TrimEnd();
        }
    }
}

```

Player.cs

```

using System.Reflection.Metadata.Ecma335;

namespace SwinAdventure
{
    public class Player : GameObject, IHAVEInventory

```

```

{
    private Inventory _inventory;

    public Player(string name, string desc): base( new string[] { "me", "inventory"}, name,
desc)
    {
        _inventory = new Inventory();
    }

    public GameObject Locate(string id)
    {
        if (AreYou(id)) return this;
        return _inventory.Fetch(id);
    }

    public override string FullDescription
    {

        get
        {
            return $"You are {Name}, {base.FullDescription}.\nYou are
carrying:\n{_inventory.ItemList}";
        }
    }

    public Inventory Inventory
    {
        get { return _inventory; }
    }
}
}

```

Bag.cs

```
namespace SwinAdventure
{
    public class Bag : Item, IHAVEInventory
    {
        private Inventory _inventory;

        public Bag(string[] ids, string name, string desc) : base(ids, name, desc)
        {
            _inventory = new Inventory();
        }

        public GameObject Locate(string id)
        {
            if (AreYou(id))
            {
                return this;
            }

            return _inventory.Fetch(id);
        }

        public override string FullDescription
        {
            get
            {
                string itemsDescription = _inventory.ItemList;

                if (string.IsNullOrEmpty(itemsDescription))
                {
```

```

        return $"{base.FullDescription}. It's empty.";
    }
    else
    {
        return $"{base.FullDescription}. In the {Name}, you can
see:\n{itemsDescription}";
    }
}
}
}

```

```

public Inventory Inventory
{
    get { return _inventory; }
}
}
}

```

TestLookCommand.cs

```

using NUnit.Framework.Legacy;
using SwinAdventure;
namespace UnitTest;

```

```

public class TestLookCommand
{
    private Player _player;
    private Item _gem;
    private Bag _bag;

```



```
private LookCommand _lookCommand;
```

```
[SetUp]
```

```
public void Setup()
```

```
{
```

```
    _player = new Player("Fred", "the mighty programmer");
```

```
    _gem = new Item(new string[] { "gem" }, "a gem", "A bright red gem");
```

```
    _lookCommand = new LookCommand();
```

```
    _player.Inventory.Put(_gem);
```

```
    _bag = new Bag(new string[] { "bag" }, "small bag", "A small leather bag");
```

```
    _player.Inventory.Put(_bag);
```

```
}
```

```
[Test]
```

```
public void TestLookAtMe()
```

```
{
```

```
    string result = _lookCommand.Execute(_player, new string[] { "look", "at", "inventory" });
```

```
    ClassicAssert.That(result, Is.EqualTo(_player.FullDescription));
```

```
}
```

```
[Test]
```

```
public void TestLookAtGem()
```

```
{
```

```
    string result = _lookCommand.Execute(_player, new string[] { "look", "at", "gem" });
```

```
    Assert.That(result, Is.EqualTo("A bright red gem"));
```

```
}
```

[Test]

public void TestLookAtUnk()

{

string result = _lookCommand.Execute(_player, new string[] { "look", "at", "ruby" });

Assert.That(result, Is.EqualTo("I can't find the ruby"));

}

[Test]

public void TestLookAtGemInMe()

{

string result = _lookCommand.Execute(_player, new string[] { "look", "at", "gem", "in",
"inventory" });

Assert.That(result, Is.EqualTo("A bright red gem"));

}

[Test]

public void TestLookAtGemInBag()

{

var gemInBag = new Item(new string[] { "gem" }, "a gem", "A red gem in the bag");

_bag.Inventory.Put(gemInBag);

string result = _lookCommand.Execute(_player, new string[] { "look", "at", "gem", "in",
"bag" });

Assert.That(result, Is.EqualTo("A red gem in the bag"));

}

```
[Test]

public void TestLookAtGemInNoBag()

{
    _player.Inventory.Take("bag"); // Remove the bag

    string result = _lookCommand.Execute(_player, new string[] { "look", "at", "gem", "in",
"bag" });

    Assert.That(_player.Inventory.HasItem("bag"), Is.False);

    // Assert.That(result, Is.EqualTo("I can't find the bag"));

}
```

```
[Test]

public void TestLookAtNoGemInBag()

{
    string result = _lookCommand.Execute(_player, new string[] { "look", "at", "gem", "in",
"bag" });

    Assert.That(result, Is.EqualTo("I can't find the gem"));

}
```

```
[Test]

public void TestInvalidLook()

{
    string result1 = _lookCommand.Execute(_player, new string[] { "look", "around" });

    Assert.That(result1, Is.EqualTo("I don't know how to look like that"));

    string result2 = _lookCommand.Execute(_player, new string[] { "hello", "105292912" });

    Assert.That(result2, Is.EqualTo("Error in look input"));

}
```

```

string result3 = _lookCommand.Execute(_player, new string[] { "look", "at", "Fred" });

Assert.That(result3, Is.EqualTo("I can't find the Fred"));

}

}

```

NUnit Console Output

